

ROS2 핵심 통신 개념

노드 · 토픽 · 서비스 · 액션 & colcon

cchyun@gmail.com

Part 1. 노드(Node)

- 노드는 로봇 시스템 내에서 특정한 단일 목적을 수행하는 개별 프로세스(실행 단위)
 - 하나의 노드 = 하나의 전문 역할
 - 레이저 거리 측정기(LiDAR)를 제어하는 노드
 - 바퀴 모터를 제어하는 노드
 - 로봇의 현재 위치를 계산하는 노드
 - 복잡한 로봇 시스템을 작고 독립적인 단위로 분리하여 설계
 - 실제 예시: 카메라 노드, 모터 제어 노드, 경로 계획 노드

- 로봇 시스템을 노드 단위로 나누어 설계해야 하는 이유
 - 장애 격리(Fault Isolation)
 - 특정 노드에 문제가 생겨도 시스템 전체가 멈추지 않음
 - 문제가 발생한 노드만 재시작하거나 교체 가능
 - 코드 재사용성
 - 한 번 만든 노드(예: LiDAR 드라이버)를 여러 프로젝트에서 재사용
 - 독립적 테스트
 - 개별 노드를 분리하여 단위 테스트 가능

- 시스템 내 노드를 구분하기 위한 식별 체계
 - 노드 이름(Node Name)
 - 시스템 내에서 각 노드를 식별하기 위한 고유한 명칭
 - 네임스페이스(Namespace)
 - 여러 대의 로봇이나 동일한 센서를 여러 개 사용할 때 이름 충돌 방지
 - '폴더'와 같은 개념으로 이름을 계층적으로 관리

```
/robot1/chatter ← robot1 네임스페이스의 chatter 노드  
/robot2/chatter ← robot2 네임스페이스의 chatter 노드
```

- 실행 중인 노드를 확인하고 제어하기 위한 명령어

명령어	설명
<code>ros2 run <패키지> <실행파일></code>	노드 실행
<code>ros2 node list</code>	현재 실행 중인 노드 목록 조회
<code>ros2 node info <노드이름></code>	특정 노드의 상세 정보 조회

- 노드를 실행하는 기본 명령어

```
ros2 run <패키지_이름> <실행파일_이름>
```

```
# 예시: turtlesim 패키지의 turtlesim_node 실행
```

```
ros2 run turtlesim turtlesim_node
```

```
# 예시: 노드 이름을 직접 지정하여 실행 (리매핑)
```

```
ros2 run turtlesim turtlesim_node --ros-args --remap __node:=my_turtle
```

- 패키지_이름 : 노드가 속한 ROS2 패키지
- 실행파일_이름 : 패키지 안에서 빌드된 실행 가능한 노드 파일
- --ros-args --remap __node:= : 실행 시 노드 이름을 원하는 이름으로 변경

- 현재 ROS2 네트워크에서 실행 중인 모든 노드를 조회

```
ros2 node list
```

```
# 출력 예시  
/turtlesim  
/teleop_turtle  
/robot1/camera_node  
/robot2/camera_node
```

- 실행 중인 노드가 없으면 아무것도 출력되지 않음
- 네임스페이스가 있는 경우 `/네임스페이스/노드이름` 형태로 표시

- 특정 노드가 어떤 토픽·서비스·액션에 연결되어 있는지 확인

```
ros2 node info <노드_이름>
```

예시

```
ros2 node info /turtlesim
```

출력 예시

/turtlesim

Subscribers: ← 이 노드가 구독 중인 토픽

 /turtle1/cmd_vel: geometry_msgs/msg/Twist

Publishers: ← 이 노드가 발행 중인 토픽

 /turtle1/pose: turtlesim/msg/Pose

Service Servers: ← 이 노드가 제공하는 서비스

 /turtle1/set_pen: turtlesim/srv/SetPen

Action Servers: ← 이 노드가 제공하는 액션

 /turtle1/rotate_absolute: turtlesim/action/RotateAbsolute

Part 2. 토픽(Topic)

- 발행자(Publisher)와 구독자(Subscriber) 구조로 데이터를 주고받는 방식
 - 발행자(Publisher): 특정 토픽에 메시지를 계속 보내는 노드
 - 구독자(Subscriber): 특정 토픽을 구독하여 메시지를 수신하는 노드
 - 발행자는 구독자가 누구인지, 잘 받았는지 확인하지 않고 계속 데이터를 보냄
 - 하나의 토픽에 여러 발행자·구독자가 동시에 참여 가능 (Many-to-Many)
 - 실제 예시: LiDAR 센서 스캔 데이터 전송, 로봇 위치(Pose) 방송

- 토픽을 사용해야 하는 상황
 - 연속적인 스트리밍 데이터
 - 센서 값처럼 실시간으로 계속 업데이트되는 데이터
 - 예: 카메라 이미지(30fps), LiDAR 스캔, IMU 데이터
 - 비동기 단방향 통신
 - 발행자와 구독자가 서로 독립적으로 동작
 - 응답을 기다리지 않아도 되는 경우
 - 다대다(Many-to-Many) 통신
 - 배터리 상태를 모든 노드에게 동시에 방송하는 경우

- 토픽 통신에 사용되는 데이터 구조(메시지 타입)

- `std_msgs/msg/String`
 - 간단한 문자열 데이터 전송용
- `std_msgs/msg/Int32`, `Float64` 등
 - 숫자형 데이터 전송용
- `sensor_msgs/msg/LaserScan`
 - LiDAR 거리 측정 데이터
- `sensor_msgs/msg/Image`
 - 카메라 이미지 데이터
- `geometry_msgs/msg/Twist`
 - 로봇 이동 속도(선속도 + 각속도) 명령

https://docs.ros2.org/latest/api/std_msgs/index-msg.html

https://docs.ros2.org/latest/api/sensor_msgs/msg/LaserScan.html

https://docs.ros2.org/latest/api/geometry_msgs/index-msg.html

- 통신 품질을 세밀하게 조정하는 QoS 설정
 - Reliability (신뢰성)
 - `Reliable`: 모든 데이터 전달을 보장 (손실 재전송)
 - `Best Effort`: 손실을 감수하고 최신 데이터 우선 전달
 - Durability (지속성)
 - `Transient Local`: 구독자가 나중에 연결되어도 이전 데이터를 보관했다가 전달
 - `Volatile`: 연결된 순간부터의 데이터만 전달
 - 활용 예시: 센서 데이터 → `Best Effort` / 로봇 설정값 → `Reliable`

- 실행 중인 토픽을 확인하고 제어하기 위한 명령어

명령어	설명
<code>ros2 topic list</code>	현재 활성화된 토픽 목록 조회
<code>ros2 topic echo <토픽이름></code>	토픽 메시지 실시간 출력
<code>ros2 topic info <토픽이름></code>	특정 토픽의 상세 정보 조회
<code>ros2 topic pub <토픽이름> <타입> <데이터></code>	토픽 메시지 직접 발행
<code>ros2 topic hz <토픽이름></code>	토픽 발행 주기(Hz) 측정

- 현재 ROS2 네트워크에서 활성화된 모든 토픽을 조회

```
ros2 topic list
```

```
# 출력 예시  
/turtle1/cmd_vel  
/turtle1/pose  
/parameter_events  
/rosout
```

```
# 메시지 타입도 함께 출력  
ros2 topic list -t
```

```
# 출력 예시 (-t 옵션)  
/turtle1/cmd_vel [geometry_msgs/msg/Twist]  
/turtle1/pose [turtlesim/msg/Pose]
```

- 퍼블리셔 또는 구독자가 존재하는 토픽만 표시됨

- 특정 토픽에 발행되는 메시지를 실시간으로 출력

```
ros2 topic echo <토픽_이름>
```

```
# 예시: 터틀심 위치 정보 실시간 확인  
ros2 topic echo /turtle1/pose
```

```
# 출력 예시  
x: 5.544444561004639  
y: 5.544444561004639  
theta: 0.0  
linear_velocity: 0.0  
angular_velocity: 0.0  
---
```

- 특정 토픽의 메시지 타입과 퍼블리셔·구독자 수를 확인

```
ros2 topic info <토픽_이름>
```

```
# 예시  
ros2 topic info /turtle1/cmd_vel
```

```
# 출력 예시  
Type: geometry_msgs/msg/Twist  
Publisher count: 1  
Subscription count: 1
```

- `-v` 옵션으로 연결된 노드 이름과 네임스페이스까지 확인 가능

터미널에서 직접 토픽 메시지를 발행

```
# 메시지 타입 확인
ros2 interface show <메시지_타입>
```

```
ros2 topic pub <토픽_이름> <메시지_타입> <데이터>
```

```
# 예시: 터틀심을 앞으로 이동
ros2 topic pub /turtle1/cmd_vel geometry_msgs/msg/Twist \
  "{linear: {x: 2.0, y: 0.0, z: 0.0}, angular: {x: 0.0, y: 0.0, z: 1.8}}"
```

옵션	설명	예시
(기본)	1Hz로 지속 발행	
<code>--once</code>	1회만 발행 후 종료	<code>ros2 topic pub --once ...</code>
<code>-r <Hz></code>	발행 주기 지정	<code>ros2 topic pub -r 10 ...</code>

- 특정 토픽의 메시지 발행 주기(frequency)를 측정

```
ros2 topic hz <토픽_이름>
```

```
# 예시: 터틀심 포즈 토픽의 발행 주기 측정
```

```
ros2 topic hz /turtle1/pose
```

```
# 출력 예시
```

```
average rate: 62.512
```

```
    min: 0.015s   max: 0.017s   std dev: 0.00047s   window: 64
```

- 센서 노드가 정상적으로 동작 중인지 빠르게 확인할 때 유용
- 예상 Hz보다 낮으면 → 노드 과부하 또는 네트워크 문제 의심

Part 3. 서비스(Service)

- 클라이언트가 요청(Request)을 보내면 서버가 처리 후 응답(Response)을 반환하는 방식
 - 토픽이 끊임없이 흐르는 "라디오 방송" 이라면
 - 서비스는 연결하고 대화하는 "전화 통화"
- 서비스를 사용해야 하는 상황:
 - 빠르게 끝나는 일회성 작업 (LED 켜기/끄기)
 - 노드에 특정 상태를 물어볼 때 (센서 현재 설정값 조회)
 - 즉각적인 결과가 필요한 계산 요청 (역기구학 계산)
- 실제 예시: 로봇 LED 제어, 센서 설정값 조회, 간단한 연산 수행

- 서비스 인터페이스는 `.srv` 파일로 정의
 - 요청(Request)과 응답(Response)을 `---` (세 개의 대시)로 구분

```
# 요청(Request) 부분
int64 a
int64 b
---
# 응답(Response) 부분
int64 sum
```

- 의미: 두 정수 `a`, `b`를 요청하면 합계 `sum`을 응답
- 주의: Python(rclpy)에서 서비스는 프로그램이 멈추지 않도록 비동기 방식(`call_async`)을 사용하는 것이 일반적

Part 4. 액션(Action)

- 목표(Goal) → 피드백(Feedback) → 결과(Result)의 3단계로 구성된 통신 방식

- **Goal:** 클라이언트가 서버에 수행할 목표를 전달
- **Feedback:** 서버가 작업 수행 중 현재 상태를 클라이언트에 전송
- **Result:** 작업이 완료되면 최종 결과를 반환
- **서비스와의 차이점:**
 - 서비스 → 즉각적인 응답 / 액션 → **장시간 실행되는 작업**
 - 작업 중 **취소(Cancel)** 가능
 - 중간 **피드백(진행률)** 확인 가능
- **실제 예시:** 목표 지점까지 내비게이션, 로봇 팔 복잡한 궤적 이동

- 로봇이 멀리 떨어진 목표 지점까지 이동하는 상황
 - 왜 서비스로는 부족한가?
 - 이동 중 몇 분이 소요될 수 있음
 - 서비스는 응답이 올 때까지 블로킹(Blocking) 상태가 됨
 - 액션이 필요한 이유:
 - 이동 중 현재 위치 피드백 을 실시간으로 수신
 - 중간에 장애물이 나타나면 이동 취소 명령 전송 가능
 - 목적지 도달 후 성공/실패 결과 수신
 - `rclpy` + `ActionClient` / `ActionServer` 로 구현

Part 5. 통신 방식 비교

구분	토픽	서비스	액션
통신 모델	발행/구독 (Many-to-Many)	요청/응답 (One-to-One)	목표/피드백/결과
시간	연속적/스트리밍	즉각적/일시적	장기적/지속적
특징	비동기, 데이터 흐름 중심	동기/비동기, 결과 확인	중간 피드백, 취소 가능
적합한 작업	센서 데이터 전송	간단한 연산, 설정 변경	주행, 로봇 팔 조작
신뢰성	QoS에 따라 다름	신뢰성 높음 (DDS 기반)	신뢰성 높음 (피드백 포함)

- 상황에 맞는 통신 방식 선택이 핵심
 - 토픽을 선택하는 경우
 - 자율주행 로봇이 LiDAR 센서로부터 1초에 30번씩 거리 데이터를 받아올 때
 - 배터리 잔량을 모든 노드에게 실시간으로 방송할 때
 - 서비스를 선택하는 경우
 - 로봇 팔의 현재 관절 각도를 한 번만 물어볼 때
 - IK(역기구학) 계산 결과를 즉시 받아야 할 때
 - 액션을 선택하는 경우
 - 로봇에게 "거실로 가서 청소해" 명령 (이동 중 위치 보고 + 취소 가능)

Part 6. colcon 빌드 시스템

- ROS2 패키지(Package)는 ROS2 코드를 조직화하는 최소 단위의 공간
 - 배포 및 공유 용이성
 - 코드를 패키지 형태로 구성하면 다른 사용자가 쉽게 빌드하고 사용 가능
 - 의존성 관리
 - 각 패키지는 자체적인 의존성 정보를 포함하여 체계적으로 관리
 - 빌드 자동화
 - `colcon`을 사용하여 여러 패키지를 한 번에 빌드하거나 특정 패키지만 선택적으로 빌드 가능

- Python 기반 ROS2 패키지 생성: `ament_python` 빌드 타입 사용

```
ros2 pkg create --build-type ament_python <package_name> \  
               --node-name <node_name>
```

- `--node-name` 옵션: 실행 가능한 Hello World 형태의 노드 파일 자동 생성
- 예시: `hello_pkg` 패키지 생성

```
ros2 pkg create --build-type ament_python hello_pkg \  
               --node-name hello_node
```


- 생성된 패키지의 상세 파일 구조

- `package.xml` — 패키지 메타 정보(이름, 버전, 의존성)를 담는 XML 파일
- `setup.py` — 패키지 설치 방법과 실행 파일(entry points)을 정의
- `setup.cfg` — `ros2 run` 명령어가 실행 파일을 찾을 수 있도록 경로 설정
- `<package_name>/` — 실제 Python 소스 코드(`.py`)와 `__init__.py`가 위치
- `resource/<package_name>` — ROS2 도구들이 패키지 식별에 사용하는 마커 파일

- `package.xml` 은 패키지의 신분증 역할을 하는 메타데이터 파일

```
<package format="3">
  <name>my_robot_pkg</name>           <!-- 패키지 고유 이름 -->
  <version>0.0.1</version>            <!-- 버전 번호 -->
  <description>My robot node</description> <!-- 요약 설명 -->
  <maintainer email="you@email.com">Name</maintainer>
  <license>Apache-2.0</license>       <!-- 오픈 소스 라이선스 -->

  <buildtool_depend>ament_python</buildtool_depend>
  <depend>roscpp</depend>             <!-- 빌드+실행 모두 필요한 의존성 -->
  <exec_depend>roslaunch</exec_depend> <!-- 실행 시에만 필요한 의존성 -->
</package>
```

- `entry_points` 는 터미널 명령어와 Python 함수를 연결해주는 설정

```
entry_points={
    'console_scripts': [
        'hello_node = hello_pkg.hello_node_file:main',
    ],
},
```

- 형식: '명령어 이름 = 패키지명.모듈명:함수명'
- 의미: `ros2 run hello_pkg hello_node` 명령어를 입력하면 `hello_pkg/hello_node_file.py` 의 `main()` 함수를 실행
- 여러 노드 등록 예시:

```
'talker = my_pkg.talker_node:main',
'listener = my_pkg.listener_node:main',
```

- `colcon build` 명령 실행 시 유용한 옵션들
 - `--packages-select <pkg_name>`
 - 전체 워크스페이스 대신 특정 패키지만 빌드
 - 빌드 시간을 단축할 때 유용

```
colcon build --packages-select hello_pkg
```

- `--symlink-install`
 - 소스 파일 복사본 대신 심볼릭 링크 생성
 - Python 스크립트 수정 시 다시 빌드하지 않아도 즉시 반영

```
colcon build --symlink-install
```

- 빌드 후 반드시 실행해야 하는 환경 설정 명령
 - `colcon build` 실행 후 `install/` 디렉토리에 환경 설정 스크립트 생성
 - `source install/setup.bash`를 실행하는 이유:
 - 현재 터미널의 환경 변수(PATH 등)에 새로 빌드된 패키지의 경로 추가
 - 이 과정 없이는 `ros2 run`이 새 패키지와 실행 파일을 인식하지 못함

```
# 빌드 완료 후
source install/setup.bash

# 이제 ros2 run 사용 가능
ros2 run hello_pkg hello_node
```

- 주의: 새로운 터미널을 열 때마다 다시 실행해야 함

- 빌드 오류 유형별 원인과 해결 방법 (1/2)

- 의존성 누락

- 원인: 필요한 패키지가 설치되지 않음
- 해결: `rosdep install --from-paths src --ignore-src -y`

- 패키지 이름 불일치

- 원인: `package.xml` 과 `setup.py` 의 이름이 다름
- 해결: 두 파일의 이름 정보가 일치하는지 확인

- 경로 문제

- 원인: 워크스페이스 루트가 아닌 곳에서 `colcon build` 실행
- 해결: 반드시 `~/Workspace/ros2_ws` (워크스페이스 루트)에서 실행

- 빌드 오류 유형별 원인과 해결 방법 (2/2)

- 커스텀 인터페이스 빌드 실패

- 원인: 커스텀 메시지/서비스를 Python 패키지 내부에 직접 정의
- 해결: 별도의 `ament_cmake` 인터페이스 패키지로 분리

Part 7. 실습 — Hello ROS2 패키지

- 워크스페이스 폴더를 만들고 패키지를 생성하는 단계

```
# 워크스페이스 폴더 생성 및 이동
mkdir -p ~/hello_ws/src
cd ~/hello_ws/src

# Python 패키지 생성 (노드 이름: hello_node)
ros2 pkg create --build-type ament_python hello_pkg \
               --node-name hello_node
```

- 생성 후 확인해야 할 파일들:

- hello_pkg/package.xml
- hello_pkg/setup.py
- hello_pkg/hello_pkg/hello_node.py

- `package.xml` 과 `setup.py` 의 기본 정보를 일치하게 수정 (권장)
 - `package.xml` 수정 항목:
 - `<description>` : 패키지 설명
 - `<maintainer email="...">` : 유지보수자 이메일
 - `<license>` : 라이선스 (예: `Apache-2.0`)
 - `setup.py` 수정 항목:
 - `maintainer`, `maintainer_email`, `description`, `license`
 - 두 파일의 값이 다르면 일부 과정에서 실패할 수 있음
 - 유지보수 관점에서 일치시키는 것을 강력 권장

- 패키지를 빌드하고 노드를 실행하는 단계

- Step 3: 빌드 실행

```
# 워크스페이스 루트로 이동
cd ~/Workspace/ros2_ws

# 심볼릭 링크 옵션으로 빌드
colcon build --symlink-install
```

- Step 4: 환경 설정 및 실행

```
# 빌드된 환경 반영
source install/setup.bash

# 노드 실행
ros2 run hello_pkg hello_node
```

- 실행 결과: `Hi from hello_pkg.` 출력 → 성공!

Part 8. 퀴즈 & 복습

- 다음 문제를 풀어보세요

1. 토픽(Topic)은 노드들이 서로의 이름을 알아야만 통신 가능한 1:1 방식이다.
2. 서비스(Service)는 내비게이션처럼 시간이 오래 걸리는 작업에 적합하다.
3. 액션(Action)은 Goal, Feedback, Result 세 가지 요소로 구성된다.
4. C++와 Python으로 작성된 노드끼리는 통신할 수 없다.

- 계속해서 풀어보세요

5. `colcon build` 명령은 워크스페이스 내의 모든 패키지를 한 번에 빌드한다.
6. 파라미터(Parameter)는 정수, 실수, 불리언, 문자열 등을 저장할 수 있다.
7. `package.xml` 파일은 패키지의 이름, 버전, 의존성 정보를 담는 메타데이터 파일이다.
8. 하나의 실행 파일(.py) 안에는 반드시 하나의 노드만 정의해야 한다.

- 다음 상황에서 적합한 통신 방식은? (토픽 / 서비스 / 액션)
 1. 자율주행 로봇이 LiDAR 센서로부터 1초에 30번씩 거리 데이터를 받아온다.
 2. 로봇 팔의 현재 관절 각도를 한 번만 물어보고 응답을 받는다.
 3. 로봇에게 "거실로 가서 청소해" 명령, 이동 중 위치 보고 + 취소 가능해야 한다.
 4. 배터리 잔량을 모든 노드에게 실시간으로 방송한다.



- 오늘 실습을 마친 후 다음 사항들을 완벽히 수행했는지 확인하세요
 - [] `ros2 pkg create --build-type ament_python` 명령어로 패키지를 생성했는가?
 - [] `colcon build` 실행 시 `--symlink-install` 옵션을 사용했는가?
 - [] 새로운 터미널을 열 때마다 `source install/setup.bash` 를 실행했는가?
 - [] `ros2 node list` 나 `rqt_graph` 도구로 노드 간 통신 구조를 시각적으로 확인했는가?